

BlockVote

Decentralized Election DApp

Submitted by: Shai Shargal
Final Course Project (90 points)
Ethereum Sepolia Deployment

Known Bugs & Limitations

Limitations

- Questionnaire selection is UI-hidden but NOT cryptographically anonymous
- Candidate ID visible in blockchain transaction calldata (intentional design)
- Admin can set election times only once (no modification after configuration)
- Vote records are permanent (no reversal mechanism)

✓ Verified Working

- 28/28 unit tests passing (Solidity)
- Merkle tree voter verification
- ERC-20 BAL token minting
- Admin access control (onlyOwner)
- Time-gating enforcement
- Results correctly gated post-election

Project Requirements (8/8 Implemented)

- ✓ Admin GUI — Manage candidates, configure election, upload voter registry
- ✓ Candidates — Add with name and 3 policy position scores (1–5 scale)
- ✓ Merkle voter registry — Voter eligibility verified via Merkle proof on-chain
- ✓ IPFS storage — Voter CSV uploaded to IPFS; CID stored in smart contract
- ✓ Election time window — Admin sets start/end times; voting strictly enforced
- ✓ Direct voting — Select candidate from dropdown; vote with Merkle proof
- ✓ Questionnaire voting — 3-slider matching (Manhattan distance) auto-selects
- ✓ ERC-20 reward — 10 BAL tokens minted automatically per vote
- ✓ Sorted results — Candidates ranked by vote count; winner identified

Architecture Overview

Frontend (React + Vite) • Web3.js client-side

↓ Reads/writes

MetaMask Wallet • Signs transactions, Sepolia

↓ Signs with private key

Election.sol + BALToken.sol • On-chain logic + ERC-20 minting

↓ Stores data

Merkle Tree + IPFS • Voter registry (CSV) on IPFS

Smart Contracts

Election.sol

- Admin-only functions • Merkle proof verification • Double-vote prevention
- Time-window enforcement • CEI pattern (safe) • Custom error types

BALToken.sol (ERC-20)

- Immutable minter address • Only Election can mint • 10 BAL per voter
- Standard ERC-20 • Prevents unauthorized minting

✓ Test Coverage: 28/28 passing

Double-voting, Merkle validation, time windows, BAL minting, results

Voter Registry & Questionnaire

Merkle Tree Voter Registry

1. Admin uploads voter CSV
2. Frontend builds Merkle tree
3. Root stored on-chain
4. CSV uploaded to IPFS
5. CID stored on-chain

Questionnaire Voting

1. Voter answers 3 questions (1–5 scale)
2. System calculates Manhattan distance
3. Closest candidate selected
4. Vote submitted (with proof)
5. Candidate revealed after vote

■■ Questionnaire result is UI-hidden pre-vote but visible on-chain in calldata

End-to-End Demo Flow

1. Admin connects wallet → Admin Dashboard
2. Admin adds 3 candidates (name + positions)
3. Admin uploads voter CSV
4. Merkle root + IPFS CID stored on-chain
5. Admin sets election window (start + end times)
6. Voter connects (must be in registry)
7. Voter selects candidate OR answers questionnaire
8. Voter receives 10 BAL tokens automatically
9. Election ends → voting window closes
10. Results displayed: winner + bar chart + ranked table

Status: All 10 steps executable. E2E flow documented in COMPLIANCE_AUDIT.md.

Deployment & Instructions

Sepolia Deployment (Verified)

Election: 0xeA97c7e23B1300ea9523A3630827C85336e2B12F

BALToken: 0xab8f347351720Ce0Fa8527b6826149da6dB950C9

Admin: 0x9D244f3b124085D9bCAfF1D77304b145BFFc749d

Installation & Testing

npm install (blockchain/ and frontend/)

npx hardhat test → 28/28 passing

npm run dev (frontend at localhost:5173)

.env: VITE_ELECTION_ADDRESS, VITE_BAL_TOKEN_ADDRESS, VITE_ADMIN_ADDRESS

GitHub: github.com/Shai-Shargal/blockchain-election-dapp